

# Lab 3: Optimization Techniques
# Advanced Computerarchitecture

(B-KUL-JLI1QS)

Reinout Annaert, Michiel Van Nieuwenhove

March 3, 2024




| | |
|---|---|
| Session Date: | 26/02/2024 |
| Class: | MElsw |
| Instructor: | Alex Marinšek |

During the third lab session, we explored the intricacies of warps, where threads are grouped and mapped to cores. Our investigation included understanding thread divergence within a warp, where a subset exhibits higher complexity. Additionally, we covered the concept of striding, enabling a thread to process multiple elements. These concepts were applied practically in an exercise centered on image inversion, conducted on the MX250 GPU with 3 Streaming Multiprocessors and 384 CUDA cores.

We developed a kernel for image inversion (`Kernel`) and, to study thread divergence, a second kernel (`Augmented Kernel`) modified the R component with a more complex formula. This led to faster processing of G and B components, leaving part of the warp idle. Subsequently, we designed a third kernel (`Augmented Kernel Reformat`) processing a reformatted data version, alternating `RR...RGGG...GBB...B` instead of `RGBRGB...RGB`. The thread diversion is implemented by checking if the thread index can be divided by three. This way, one in three threads calculates the other formula for the R component. To operate on the colors in order instead of by index, a new thread-ID is calculated by taking into account the index and the color-channel. With this, a new position is calculated and used to operate on another index of the image.

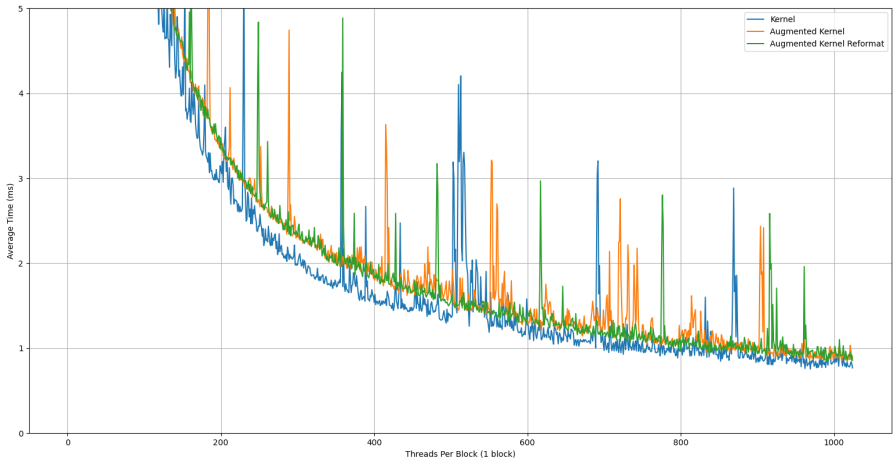
All three kernels incorporate striding, with correctness verified using fewer threads than input data elements. The image inversion remained correct, thanks to striding. To implement this, instead of one thread operating on one pixel, a loop is used. Every thread starts at the pixel equivalent to their ID. Then, they take a step with size equal to the total amount of threads, and execute the necessary operation on that index as well. When they reach an index bigger than the image size they stop. This ensures that all pixels get operated on, and the workload is divided between threads. To put this concretely, the for-loop can look as follows:

$$\text{for (int idx} = \text{initialIndex; idx} < \text{width} * \text{height} * 3; \text{idx} \mathrel{+}= \text{threadCount)}$$

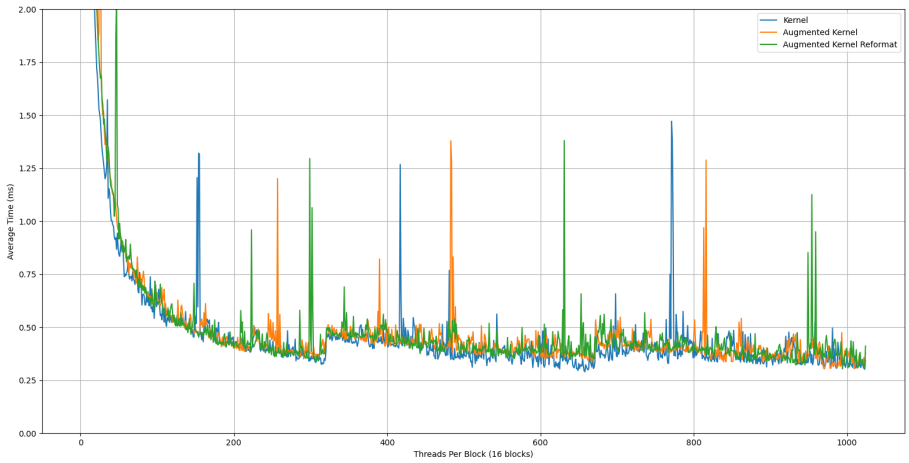
with the initial index being: *blockIdx.x * blockDim.x + threadIdx.x*, like in the other CUDA labs. The 'threadCount'-variable can be calculated as: *gridDim.x * blockDim.x*.

In conclusion, we evaluated execution times concerning thread count and warp occupancy. We conducted 100 measurements for each configuration, recording average execution times. Figures 1a and 1b illustrate measurements utilizing one and sixteen blocks, respectively. We see that the initial kernel performs in general better than the ones with a harder operation on the R components. This is because with the harder operation the threads need longer to calculate this. The kernel that calculates the colors in order instead of the pixels performs a bit better than the other one. If the pixels are calculated in order a part of the warp remains inactive, because of the difference in workload between the threads. By calculating the colors in order, the threads within a warp have a much more similar workload.

The GitHub repository containing the source code can be found at *Github Repository*




(a) 1 Block




(b) 16 Blocks

Figure 1: Influence of Threads per Block on Execution Times